Contents

Purpose 2

Roles and Responsibilities 2

Definitions 2

Procedure 2

- Pulling Down Information 2
- Pushing Code Changes 3
- Committing Code Changes onto the Main Branch 4
- Handling Merge Conflicts 5
- Deleting a Local Branch 6
- Using Git Ignore 6

Additional Resources 7

2026 Generic Company, Inc. All rights reserved.

Purpose

This document is for Developers who are working with Git and Git-related processes. Here you'll find the steps involved in pulling, pushing, and committing code using Git, as well as handling merge conflicts.

Roles and Responsibilities

Developer: Pulls code from the `main` branch, makes updates, and pushes the updates back to `main`, using Git and Azure DevOps.

Definitions

Azure DevOps: A Microsoft service Developers use to facilitate end-to-end software development.

Git: A repository that Developers use to share common code and make updates to the code files.

Stash: A function in Visual Studio Code that enables you to move modules that contain your code updates onto a local device.

Procedure

Pulling Down Information

Whenever you update code within the `main` branch, you must pull down the branch's information first. To do this, follow the steps below.

1. Log into Visual Studio Code and, within the *Team Explorer* pane, select **Git Changes**. This takes you to the *Git Changes* pane, which displays the `main` branch.
2. Click on the (*pull*) icon next to the `main` dropdown. This updates your local files with the code changes that users made since the last time you tried to fetch from the `main` branch.

   Tip: If you'd like to see code changes that users have made without updating your local files, click on the **Fetch** button. Then select the **outgoing / incoming** link.

3. If the pull is successful, the message "**Repository updated to commit**" appears.

2026 Generic Company, Inc. All rights reserved.

Pushing Code Changes

After you've pulled the `main` branch's code and made your updates, you must create a user branch and use it to push your changes.

Follow the steps below to create the user branch and push the code.

1. Select the **`main`** dropdown and click **New Branch**.
2. Enter a name for the user branch you want to create (this user branch is built off the `main` branch). The typical naming convention for a user branch is *users/[FirstNameInitialLastName]/[branch_description]*. For example, "**users/jdoe/HDV_Demo**".
3. Click **Create**. This creates the branch locally. Once you've updated the code in your local branch, you'll see the changes in the *Git Changes* tab within the pane on the right-hand side of the screen.
4. Hover your cursor over the file you want to update and click on the + (*Staging*) icon next to it. The folder then appears under *Staged Changes* within the *Git Changes* pane. Any content under *Staged Changes* is set to be pushed to the server.

   Tip: If you'd like to remove an item from the *Staged Changes* section, hover your cursor over the item and select the — (*Unstage*) icon, or right click on the item and select **Unstage**.

5. In the *Enter a message* textbox, write a description about your update. This helps other Developers understand what you've changed. Use the following convention for your entries: *[Client Initials] – [Jira Ticket Number: Description]*. For example, "**HDV – CS-00000: This adds extra logging for the copy API**."
6. Click on the **Commit Staged** button, or select the ▾ (down arrow) icon next to *Commit Staged*, which allows you to choose one of the following options:
   i. **Commit Staged**: Moves the staged item to "outgoing", meaning that it's ready to be pushed.
   ii. **Commit Staged and Push**: Moves the staged item to "outgoing" and pushes it to the server.
   iii. **Commit Staged and Sync**: This option is not recommended since it can lead to merge conflicts.

2026 Generic Company, Inc. All rights reserved.

7. Select the **outgoing / incoming** link. This shows you the item that's ready to be pushed as well as the *Local History*.
8. Within the *Git Changes* pane, select the (push) icon. This pushes the outgoing changes to the server.
9. Open Azure DevOps and, in the side pane, click **Repos→Branches**. A screen appears, showing you the branches that are associated with your account. You should now be able to see the branch you created within this screen.
10. Click on the branch and go to the **History** tab. The item(s) you pushed will appear on the Commit table.

Committing Code Changes onto the `Main` Branch

Once you've pushed your code update, select the **Create a Pull Request** button that appears within the *Git Changes* pane. This allows you to merge your branch with the `main` branch.

You can also merge your user branch through Azure DevOps by following the steps below:

1. Open Azure DevOps and go to **Repos→Pull requests**.
2. In the *Pull requests* screen, select the **New pull request** button.
3. Select the branch dropdown in the top left-hand corner and click on the branch that you'd like to merge.
4. Ensure that `main` is selected for the "into" branch.
5. **Optional**: Select the **Files** tab to review the changes you've made. Content you've added is highlighted in green, and anything you removed is red.
6. **Optional**: You can select the **Commits** tab to see details about the commit.
7. Within the *Overview* tab, provide a **Title** and **Description** and click **Create**.
8. **Optional** (but recommended for newer Developers): In the *Reviewers* section on the right-hand side of the *Overview* screen, select the **Add** dropdown to assign an optional reviewer or required reviewer.
    a. Use the **Required** and/or **Optional** text boxes to view and select reviewers.

    > **Note**: An optional reviewer can review your change request, but their approval isn't required to complete the pull request merge. However, a required reviewer must give approval for the merge to complete.

2026 Generic Company, Inc. All rights reserved.

9. Click on the **Set auto-complete button** in the top right-hand side of the screen.
10. In the *Enable automatic completion* pane, use the **Merge Type** dropdown to select a type of commit. The *Squash commit* is recommended because it condenses multiple commits, making it more readable.
11. In the *Post-completion* options, activate **Delete [branch name] after merging**. This deletes the folder you pushed after it merges onto the `main` branch.
12. Click **Set auto-complete**.
13. To verify that your content has merged, go to **Repos→Branches** and select **`main`**.
    a. Within the *Files* screen's **History** tab, you should be able to see the item you pushed.

Handling Merge Conflicts

In Visual Studio Code, if you click on the (*pull*) icon within the *Git Changes* pane, and you see the following error, then a merge conflict has occurred.

If you encounter a merge conflict, follow the procedure below to resolve the error:

Note: There are multiple methods for resolving a merge conflict, however, this document demonstrates the recommended approach.

1. Click on the **Output** link within the error message. This displays the *Output* window on the bottom of your screen. This window reveals the problem that's occurring.
2. Create a *Stash* of all your local content by doing the following:
    a. Within the *Git Changes* pane, select the (*down arrow*) icon next to the **Commit All** button.
    b. Select **Stash All (--include-untracked)**. Your stash should appear within the *Git Changes* pane under the *Stashes* section. You can click on the stash to see the modules that have been moved to your local device.
3. Click on the (*pull*) icon within the *Git Changes* pane. This should result in a successful pull from the `main` branch, where the change is committed to `main`.
4. Right click on your stash in the *Git Changes* pane and click **Apply→Apply All as Unstaged**.

2026 Generic Company, Inc. All rights reserved.

> **Note**: *Apply* and *Pop* serve similar functions, but there's a key difference between the two: *Apply* pushes your stashed code changes and preserves the stash afterwards, while *Pop* deletes the stash after it pushes the code changes.

5. The *Unmerged Changes* section appears. Select the item that's in this section.
6. View the *Incoming* and *Current* code on the left-hand side of the screen, which shows you the conflicts that are occurring. The incoming code displays what you're trying to merge in, and the current code represents what currently exists on the file.

   > **Tip**: Click on the arrows next to the conflict counter to cycle through conflicts.

7. Within the *Incoming* view, select lines of code that you want to merge and activate the ☐ (*checkbox*) icon beside the code. This adds the code to the *Results* views.

   > **Note**: You can also activate the ☐ (*checkbox*) icon next to code in the *Current* view as well.

8. Once you've reviewed all the conflicts, click the **Accept Merge** button.
9. Your content should appear in the *Changes* section. You can now stage and push your modules as you see fit. For information on staging and pushing modules, see this document's Pushing Code Changes section.

Deleting a Local Branch

After you use a local branch to push a code update, source control deletes the branch automatically in Azure DevOps. However, it doesn't remove the branch from Visual Studio Code. You must do this manually.

To remove your local branch from Visual Studio Code, do the following.

1. Within the *Git Changes* pane, select the **`main`** dropdown and right-click on the local branch that you used to push your code change.
2. Click **Delete**.

Using Git Ignore

If you have a file that you use for your own testing purposes and you don't intend to push it, you can use Visual Studio Code's *Git Ignore* function. This keeps a file from moving to the *Changes* section so that it doesn't accidentally get pushed.

2026 Generic Company, Inc. All rights reserved.

To use *Git Ignore*, navigate to the *Git Changes* pane and right-click on the file that you don't want to push. Then select **Ignore this local item**.

This automatically generates a *.gitignore* file. When you click on it, it shows the line of code that prevents the system from adding the selected file to the *Changes* section.

Additional Resources

For additional information on Git, visit How to Use Git? Tutorials, Workflows & Commands | Atlassian.

2026 Generic Company, Inc. All rights reserved.